My Notes for Python Libraries

Ioanna Stamou

Contents

Chapter 1

Introduction

This document contains short notes about the Python libraries. Each chapter introduces a library, explains what it is mainly used for, and provides small examples.

1.1 Overview of Libraries

- **Pandas:** data processing and data analysis with tables (DataFrames)
- **NumPy:** numerical computing, arrays, and mathematical operations
- **SciPy:** scientific computing (ODE solving, integration, optimization, curve fitting)
- **Matplotlib:** visualization and plotting (line plots, histograms, scatter plots, etc.)
- **Requests:** sending HTTP requests to APIs (GET, POST, etc.)
- **FastAPI:** building backend APIs (server-side endpoints)
- **PyTest:** testing Python code (unit tests, exception tests)
- **Scikit-learn:** machine learning routines (preprocessing, training, evaluation)
- **PyTorch:** deep learning framework for neural networks, automatic differentiation, and GPU-accelerated tensor computation
- **PySpark:** big data processing using Apache Spark (distributed DataFrames and SQL)
- **Pathlib:** python library for handling path
- **xarray:** data processing library for multi-dimentional labed data
- **socket:** low-level network communication
- **pyserial:** serial communication with hardware devices
- **InfluxDB client:** writing and querying time-series data for monitoring/metrics
- **Paramiko:** SSH connections and remote command execution
- **subprocess:** running external system commands from Python

These notes are meant to be a practical reference: short explanations, clean examples, and the most common routines.

Chapter 2

Pandas

Pandas is a Python library for **data processing and data analysis**. It provides high-level data structures that make working with tabular data easy and intuitive.

2.1 Importing Pandas

```
import pandas as pd
```

The alias `pd` is a community standard and is used in almost all pandas code.

2.2 Core Data Structures

2.2.1 Series

A **Series** is a one-dimensional labeled array. It is similar to a column in a table.

```
s = pd.Series([10, 20, 30])
```

2.2.2 DataFrame

A **DataFrame** is a two-dimensional table made of rows and columns. It is the most commonly used pandas structure.

```
df = pd.DataFrame({
    "A": [1, 2, 3],
    "B": [4, 5, 6]
})
```

2.3 Basic Operations

2.3.1 Insert and Drop Columns

```
df["C"] = [7, 8, 9]       # insert column
df = df.drop("B", axis=1)  # drop column
```

`axis=1` refers to columns, while `axis=0` refers to rows.

2.4 Reading and Writing CSV Files

```
df = pd.read_csv("data.csv")
df.to_csv("output.csv", index=False)
```

CSV files are one of the most common formats for tabular data.

2.5 Indexing and Selection

2.5.1 Using `iloc`

`iloc` is used for **integer-based** indexing.

```
df.iloc[0]          # first row
df.iloc[0, 1]       # row 0, column 1
df.iloc[:, 0]       # all rows, first column
```

2.6 Basic Statistics

```
df.mean()
df.mode()
df.sum()
df.max()
df.min()
```

These functions operate column-wise by default.

2.7 Unique Values

```
df["A"].unique()
df["A"].nunique()
```

`unique()` returns the values, while `nunique()` returns how many unique values exist.

Chapter 3

NumPy

NumPy (Numerical Python) is a core Python library for **numerical computing**.
It provides:

- Fast numerical operations
- Multi-dimensional arrays
- Mathematical and statistical functions

NumPy is the foundation on which pandas and many scientific libraries are built.

3.1 Importing NumPy

```
import numpy as np
```

The alias `np` is the standard convention used in almost all NumPy code.

3.2 NumPy Arrays

A **NumPy array** is a fixed-size, homogeneous collection of numbers. Unlike Python lists, all elements have the same data type.

```
a = np.array([1, 2, 3, 4])
```

3.2.1 Array Dimensions

```
a.ndim      # number of dimensions
a.shape     # size along each dimension
a.size      # total number of elements
```

Arrays can be 1D (vectors), 2D (matrices), or higher-dimensional.

3.3 Creating Arrays

```
np.zeros(5)
np.ones((2, 3))
np.arange(0, 10, 2)
np.linspace(0, 1, 5)
```

- `zeros`, `ones`: initialize arrays
- `arange`: integer ranges
- `linspace`: evenly spaced values

3.4 Indexing and Slicing

```
a[0]        # first element
a[1:3]      # slice
a[-1]       # last element
```

```
b = np.array([[1, 2, 3],
              [4, 5, 6]])

b[0, 1]     # row 0, column 1
b[:, 0]     # all rows, first column
```

Indexing in NumPy is zero-based and very similar to pandas `iloc`.

3.5 Array Operations

3.5.1 Element-wise Operations

```
a + 2
a * 3
a ** 2
```

Operations are applied **element-wise**, without explicit loops.

3.5.2 Array-to-Array Operations

```
x = np.array([1, 2, 3])
y = np.array([4, 5, 6])

x + y
x * y
```

3.6 Mathematical Functions

```
np.mean(a)
np.sum(a)
np.max(a)
np.min(a)
np.std(a)
```

These functions are optimized and much faster than manual Python loops.

3.7 Boolean Indexing

```
a[a > 2]
```

Boolean indexing allows filtering arrays using conditions. This concept appears frequently in pandas as well.

3.8 Linear Algebra (Very Common)

NumPy provides many useful linear algebra tools (vectors, matrices, systems of equations). They are used everywhere in data science, physics, and machine learning.

```
A = np.array([[2, 1],
              [1, 3]])
b = np.array([1, 2])

np.dot(A, b)           # matrix-vector product
np.linalg.det(A)       # determinant
np.linalg.inv(A)       # inverse (if it exists)
x = np.linalg.solve(A, b)   # solve A x = b
```

`np.linalg.solve(A, b)` is preferred over computing the inverse explicitly. It is more stable and usually faster.

3.9 Random Numbers

Random numbers are used in simulation, statistics, and machine learning. NumPy provides a modern random generator through `np.random`.

```
rng = np.random.default_rng(seed=42)

rng.random(5)                # 5 random floats in [0, 1)
rng.integers(0, 10, size=5) # random integers
rng.normal(loc=0, scale=1, size=5)  # normal distribution
```

Using `default_rng` is the recommended modern approach.

3.10 Useful Array Transformations

```
a = np.array([1, 2, 3, 4, 5, 6])

a.reshape((2, 3))      # change shape
a.flatten()            # make 1D copy
a.T                    # transpose (useful for 2D arrays)
```

`reshape` is extremely common when preparing data for ML models.

3.11 Broadcasting (Super Important Concept)

Broadcasting is NumPy's rule system that lets arrays of different shapes work together in operations (without manual loops).

```
a = np.array([1, 2, 3])
b = np.array([[10],
              [20]])

a + b
```

Here, `a` is automatically "stretched" across rows to match `b`. This is one of the reasons NumPy code can be very short and fast.

3.12 Fitting with NumPy (`polyfit`)

NumPy can perform simple polynomial fitting using `np.polyfit`. For general curve fitting (any function), SciPy is typically used.

```
x = np.array([0, 1, 2, 3])
y = np.array([1, 2, 1, 3])

coeffs = np.polyfit(x, y, deg=2)  # quadratic fit
p = np.poly1d(coeffs)

p(1.5)   # evaluate fitted polynomial at x=1.5
```

`coeffs` contains the polynomial coefficients. `poly1d` creates a polynomial function object.

Chapter 4

SciPy (Numerical Methods Beyond NumPy)

SciPy is built on top of NumPy. It provides advanced scientific computing tools such as:

- Solving ODEs (differential equations)
- Numerical integration
- Optimization and curve fitting

4.1 Importing SciPy

```
from scipy import integrate
from scipy import optimize
```

SciPy is organized into submodules. For example, `integrate` for ODEs/integrals and `optimize` for fitting.

4.2 Solving Integrals

4.2.1 Single Integral (quad)

`integrate.quad` numerically computes:

$$\int_a^b f(x)\,dx$$

```
import numpy as np
from scipy import integrate

f = lambda x: np.sin(x)

result, error = integrate.quad(f, 0, np.pi)
result
```

`quad` returns the integral result and an estimate of numerical error.

4.2.2 Integrating Discrete Data (`trapz`)

```
x = np.linspace(0, np.pi, 100)
y = np.sin(x)

np.trapz(y, x)    # trapezoidal rule
```

`np.trapz` is useful when you already have sampled data points.

4.3 Solving ODEs (Differential Equations)

ODE solving is usually done with `integrate.solve_ivp`. It solves problems like:

$$\frac{dy}{dt} = f(t, y), \quad y(t_0) = y_0$$

```
import numpy as np
from scipy.integrate import solve_ivp

def dydt(t, y):
    return -2 * y    # dy/dt = -2y

t_span = (0, 5)
y0 = [1]

sol = solve_ivp(dydt, t_span, y0, t_eval=np.linspace(0, 5, 50))

sol.t    # time points
sol.y[0] # solution y(t)
```

`solve_ivp` is very flexible and can handle many ODE systems. The output `sol.y` is shaped as (num_equations, num_time_points).

4.4 Curve Fitting (General Fit)

For fitting a custom function to data, SciPy provides `optimize.curve_fit`. It estimates parameters by minimizing least-squares error.

```
import numpy as np
from scipy.optimize import curve_fit

def model(x, a, b):
    return a * np.exp(b * x)

x = np.array([0, 1, 2, 3])
y = np.array([1.0, 2.7, 7.4, 20.1])
```

```
params, covariance = curve_fit(model, x, y)

params  # [a, b]
```

`params` contains the best-fit parameters. `covariance` gives uncertainty information about the fitted parameters.

4.5 Optimization (Finding Min/Max)

Optimization means finding a value of x that minimizes or maximizes a function. SciPy provides many tools, including `minimize`.

```
import numpy as np
from scipy.optimize import minimize

f = lambda x: (x - 3)**2 + 1

result = minimize(f, x0=0)

result.x   # location of minimum
result.fun # minimum value
```

`x0` is the initial guess. Some optimization algorithms need a good starting point.

Chapter 5

Matplotlib

Matplotlib is a Python library used for creating static, animated, and interactive visualizations.
It is the most fundamental plotting library in Python.

5.1 Importing Matplotlib

```
import matplotlib.pyplot as plt
```

5.2 Basic Plot

```
x = [1, 2, 3]
y = [4, 5, 6]

plt.plot(x, y)
plt.xlabel("X axis")
plt.ylabel("Y axis")
plt.title("Simple Line Plot")
plt.show()
```

`plt.show()` renders the plot. Without it, the figure may not appear.

5.3 Multiple Lines on the Same Plot

Matplotlib allows plotting multiple datasets on the same figure. Each call to `plt.plot()` adds a new line.

```
x = [1, 2, 3]
y1 = [1, 4, 9]
y2 = [1, 2, 3]

plt.plot(x, y1, label="x squared")
plt.plot(x, y2, label="x linear")
plt.legend()
plt.show()
```

`plt.legend()` displays labels defined using the `label` argument.

5.4 Line Styles and Colors

```
plt.plot(x, y1, color="blue", linestyle="--", marker="o")
plt.plot(x, y2, color="red", linestyle="-")
plt.show()
```

Common style options:

- `color`: line color
- `linestyle`: solid, dashed, dotted
- `marker`: points on the line

5.5 Scatter Plots

Scatter plots are used to visualize relationships between variables. They are very common in data analysis.

```
plt.scatter(x, y1)
plt.xlabel("X")
plt.ylabel("Y")
plt.title("Scatter Plot Example")
plt.show()
```

5.6 Bar Plots

Bar plots are useful for comparing discrete categories.

```
categories = ["A", "B", "C"]
values = [10, 15, 7]

plt.bar(categories, values)
plt.ylabel("Value")
plt.title("Bar Plot")
plt.show()
```

5.7 Histograms

Histograms show the distribution of numerical data. They are often used in statistics and exploratory data analysis.

```
import numpy as np

data = np.random.normal(0, 1, 1000)

plt.hist(data, bins=30)
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.title("Histogram")
plt.show()
```

The `bins` parameter controls how many bars the histogram has.

5.8 Subplots

Subplots allow multiple plots in the same figure. They are useful for comparing results side by side.

```
fig, axes = plt.subplots(1, 2)

axes[0].plot(x, y1)
axes[0].set_title("Plot 1")

axes[1].plot(x, y2)
axes[1].set_title("Plot 2")

plt.show()
```

`axes` is an array of plotting areas. Each axis behaves like its own mini-plot.

5.9 Figure Size and Resolution

```
plt.figure(figsize=(6, 4), dpi=100)
plt.plot(x, y1)
plt.show()
```

`figsize` controls the size in inches. `dpi` controls resolution (dots per inch).

5.10 Saving Figures

Plots can be saved to files instead of (or in addition to) being shown. This is essential for reports and publications.

```
plt.plot(x, y1)
plt.savefig("my_plot.png")
plt.show()
```

Always call `savefig` before `show` to avoid empty images.

5.11 Why Matplotlib Is Important

- It is the foundation of most Python visualization libraries
- Almost all scientific Python code uses it
- Understanding Matplotlib makes Seaborn easier
- It is widely accepted in academic and professional work

Chapter 6

Requests and FastAPI

Modern Python applications often communicate over the network. Two key libraries are:

- **requests**: for sending HTTP requests (client-side)
- **FastAPI**: for building APIs (server-side)

6.1 Requests

requests is a simple and powerful library for making HTTP requests (GET, POST, PUT, DELETE).

6.1.1 Importing Requests

```
import requests
```

6.1.2 GET Request

```
response = requests.get("https://api.example.com/data")

response.status_code
response.json()
```

`response.json()` parses JSON responses into Python dictionaries.

6.1.3 POST Request

```
data = {"name": "Alice", "age": 30}

response = requests.post(
    "https://api.example.com/users",
    json=data
)
```

6.2 FastAPI

FastAPI is a modern Python framework for building fast, type-safe web APIs. It is widely used in backend and ML services.

6.2.1 Basic API

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def read_root():
    return {"message": "Hello World"}
```

6.2.2 Path Parameters

```
@app.get("/items/{item_id}")
def read_item(item_id: int):
    return {"item_id": item_id}
```

FastAPI automatically validates types and generates documentation.

Chapter 7

PyTest

PyTest is a testing framework for Python. It allows writing simple, readable tests for functions and modules.

7.1 Basic Test

```
def add(a, b):
    return a + b

def test_add():
    assert add(2, 3) == 5
```

7.2 Running Tests

```
pytest
```

PyTest automatically discovers files starting with `test_`.

7.3 Testing Exceptions

```
import pytest

def test_error():
    with pytest.raises(ValueError):
        int("abc")
```

Chapter 8

Scikit-learn (Machine Learning Routines)

Scikit-learn is the most widely used Python library for **classical machine learning**. It provides:

- Data preprocessing tools
- Supervised learning algorithms
- Unsupervised learning algorithms
- Model evaluation utilities

8.1 Importing Scikit-learn

```
import numpy as np
from sklearn import datasets
```

Scikit-learn is organized into submodules such as `model_selection`, `preprocessing`, and `metrics`.

8.2 Typical Machine Learning Workflow

Most scikit-learn workflows follow these steps:

1. Load data
2. Split into train and test sets
3. Preprocess data
4. Train model
5. Evaluate model

8.3 Train-Test Split

```
from sklearn.model_selection import train_test_split

X = np.array([[1], [2], [3], [4]])
y = np.array([0, 0, 1, 1])

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)
```

The test set is used only for evaluation, never for training.

8.4 Data Preprocessing

8.4.1 Feature Scaling

Many ML models require features to be on similar scales.

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_train)
```

`fit` learns parameters from data, `transform` applies the transformation.

8.4.2 Encoding Categorical Variables

```
from sklearn.preprocessing import OneHotEncoder

encoder = OneHotEncoder(sparse=False)
encoded = encoder.fit_transform([["red"], ["blue"], ["red"]])
```

8.5 Supervised Learning

8.5.1 Linear Regression

Linear regression is used for predicting continuous values.

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X_train, y_train)

predictions = model.predict(X_test)
```

8.5.2 Logistic Regression

Logistic regression is used for binary classification.

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression()
model.fit(X_train, y_train)

predictions = model.predict(X_test)
```

8.5.3 K-Nearest Neighbors (KNN)

```
from sklearn.neighbors import KNeighborsClassifier

model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_train, y_train)

predictions = model.predict(X_test)
```

8.5.4 Decision Trees

```
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier(max_depth=3)
model.fit(X_train, y_train)
```

Decision trees are intuitive but can easily overfit.

8.5.5 Random Forests

Random forests combine many decision trees to improve performance.

```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(n_estimators=100)
model.fit(X_train, y_train)
```

8.6 Unsupervised Learning

8.6.1 K-Means Clustering

Clustering groups data points without labels.

```
from sklearn.cluster import KMeans

model = KMeans(n_clusters=2)
clusters = model.fit_predict(X)
```

8.6.2 Principal Component Analysis (PCA)

PCA reduces dimensionality while preserving variance.

```
from sklearn.decomposition import PCA

pca = PCA(n_components=2)
X_reduced = pca.fit_transform(X)
```

8.7 Model Evaluation

8.7.1 Accuracy

```
from sklearn.metrics import accuracy_score

accuracy = accuracy_score(y_test, predictions)
```

8.7.2 Confusion Matrix

```
from sklearn.metrics import confusion_matrix

confusion_matrix(y_test, predictions)
```

8.7.3 Classification Report

```
from sklearn.metrics import classification_report

print(classification_report(y_test, predictions))
```

The classification report includes precision, recall, and F1-score.

8.8 Pipelines

Pipelines chain preprocessing and modeling steps safely. They help avoid data leakage.

```
from sklearn.pipeline import Pipeline

pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("model", LogisticRegression())
])

pipe.fit(X_train, y_train)
pipe.predict(X_test)
```

8.9 Cross-Validation

Cross-validation evaluates models more reliably than a single split.

```
from sklearn.model_selection import cross_val_score

scores = cross_val_score(model, X, y, cv=5)
scores.mean()
```

8.10 Why Scikit-learn Is Important

- Industry standard for classical ML
- Simple and consistent API
- Excellent documentation
- Ideal for rapid prototyping and research

Chapter 9

PyTorch (Deep Learning Framework)

PyTorch is one of the most widely used libraries for deep learning and neural networks.
It provides:

- GPU-accelerated tensor operations
- Automatic differentiation
- Neural network modules
- Optimizers and training utilities

It is heavily used for:

- computer vision
- language models
- reinforcement learning
- scientific machine learning

9.1 Installing PyTorch

Install PyTorch using pip:

```
pip install torch
```

Check installation:

```
import torch
print(torch.__version__)
```

9.2 Importing PyTorch

```
import torch
import torch.nn as nn
import torch.nn.functional as F
```

Typical aliases:

- torch → main library
- nn → neural network modules
- F → functional utilities

9.3 Tensors

Tensors are the core data structure of PyTorch. They are similar to NumPy arrays but support GPU acceleration.

Create tensors:

```
x = torch.tensor([1,2,3])
```

Random tensors:

```
torch.rand(3)
torch.randn(3)
```

Zeros and ones:

```
torch.zeros(5)
torch.ones(5)
```

9.4 Tensor Shape and Properties

```
x.shape
x.ndim
x.dtype
```

Example:

```
x = torch.tensor([[1,2],[3,4]])

x.shape → (2,2)
x.ndim → 2
```

9.5 Tensor Operations

Element-wise operations:

```
x + 2
x * 3
x ** 2
```

Tensor operations:

```
a = torch.tensor([1,2,3])
b = torch.tensor([4,5,6])

a + b
a * b
```

9.6 Matrix Multiplication

Very common in neural networks.

```
A = torch.tensor([[1,2],[3,4]])
B = torch.tensor([[5,6],[7,8]])

torch.matmul(A,B)
```

9.7 Automatic Differentiation (Autograd)

PyTorch automatically computes gradients.

```
x = torch.tensor(3.0, requires_grad=True)

y = x**2

y.backward()

print(x.grad)
```

Output:

```
6
```

because:

```
d(x²)/dx = 2x
```

9.8 Neural Network Modules

PyTorch provides many layers.

Common layers:

```
nn.Linear
nn.Embedding
nn.Conv2d
nn.LSTM
```

Example:

```
layer = nn.Linear(10,5)
```

This maps:

```
10 inputs → 5 outputs
```

9.9 Building a Neural Network

Neural networks are defined as classes.

```
class SimpleModel(nn.Module):

    def __init__(self):
        super().__init__()

        self.linear = nn.Linear(10,1)

    def forward(self, x):

        output = self.linear(x)

        return output
```

9.10 Forward Pass

The forward() method defines how data flows through the model.

```
model = SimpleModel()

x = torch.rand(1,10)

y = model(x)
```

9.11 Loss Functions

Loss measures how wrong the model prediction is.
Common losses:

```
nn.MSELoss
nn.CrossEntropyLoss
nn.BCELoss
```

Example:

```
loss_fn = nn.MSELoss()
loss = loss_fn(predictions, targets)
```

9.12 Optimizers

Optimizers update model weights.
Common optimizers:

```
SGD
Adam
RMSprop
```

Example:

```
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
```

9.13 Training Loop

The standard PyTorch training loop:

```
for epoch in range(epochs):

    optimizer.zero_grad()

    predictions = model(X)

    loss = loss_fn(predictions, y)

    loss.backward()

    optimizer.step()
```

Explanation:

```
zero_grad() → clear gradients
forward pass → compute predictions
loss → compute error
backward() → compute gradients
step() → update weights
```

9.14 Embedding Layers

Embeddings convert token IDs into vectors.
Used in language models.

```
embedding = nn.Embedding(vocab_size, embedding_dim)
```

Example:

```
token_id = 5
→ vector representation
```

9.15 Example: Tiny Language Model

Example architecture used in the tiny language model.

```
class TinyLanguageModel(nn.Module):

    def __init__(self, vocab_size, embed_dim):

        super().__init__()

        self.embedding = nn.Embedding(vocab_size, embed_dim)

        self.linear = nn.Linear(embed_dim, vocab_size)

    def forward(self, input_ids):
```

```
        x = self.embedding(input_ids)

        x = x.mean(dim=1)

        logits = self.linear(x)

        return logits
```

Pipeline:

```
tokens
↓
embedding
↓
sentence vector
↓
linear layer
↓
token probabilities
```

9.16 Generating Predictions

After training, we can generate predictions.

```
with torch.no_grad():

    logits = model(input_ids)

    probs = torch.softmax(logits, dim=-1)

    next_token = torch.argmax(probs)
```

9.17 Saving and Loading Models

Save model:

```
torch.save(model.state_dict(), "model.pt")
```

Load model:

```
model.load_state_dict(torch.load("model.pt"))
```

9.18 Why PyTorch Is Important

PyTorch is widely used because:

- simple Pythonic design
- strong research community
- flexible dynamic computation graphs
- widely used in modern AI systems

Frameworks like GPT, LLaMA, Stable Diffusion, and many research models are built with PyTorch.

Chapter 10

PySpark

PySpark is the Python interface to **Apache Spark**, a distributed computing framework used for processing large datasets.
PySpark allows Python users to leverage Spark's speed and scalability.

10.1 What Is Spark?

Apache Spark is designed for:

- Distributed data processing
- In-memory computation
- Large-scale data analytics

It is commonly used when data is too large to fit in memory on a single machine.

10.2 Starting a Spark Session

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("MyApp") \
    .getOrCreate()
```

The `SparkSession` is the main entry point to PySpark.

10.3 Creating DataFrames

```
data = [(1, "Alice"), (2, "Bob"), (3, "Charlie")]
columns = ["id", "name"]

df = spark.createDataFrame(data, columns)
df.show()
```

Spark DataFrames are similar to pandas DataFrames, but they operate in a distributed way.

10.4 Reading Data from Files

```
df = spark.read.csv(
    "data.csv",
    header=True,
    inferSchema=True
)
```

Spark supports CSV, JSON, Parquet, ORC, and many other formats.

10.5 Basic DataFrame Operations

```
df.select("name").show()
df.filter(df["id"] > 1).show()
df.groupBy("name").count().show()
```

10.6 Spark Transformations vs Actions

Spark operations are divided into:

- **Transformations**: define what to do (lazy)
- **Actions**: trigger computation

```
df_filtered = df.filter(df["id"] > 1)  # transformation
df_filtered.show()                     # action
```

10.7 Spark SQL

```
df.createOrReplaceTempView("people")

spark.sql("""
    SELECT name, COUNT(*)
    FROM people
    GROUP BY name
""").show()
```

Spark SQL allows querying DataFrames using SQL syntax.

10.8 Why Use PySpark?

- Handles very large datasets
- Scales across clusters
- Faster than traditional disk-based systems
- Integrates well with Hadoop

Chapter 11

Pathlib

Pathlib is a Python library used for **handling file paths in a clean and safe way**. It replaces traditional string-based paths with object-oriented paths.

11.1 Importing Pathlib

```
from pathlib import Path
```

`Path` is the main class used to create and manipulate file and directory paths.

11.2 Creating Paths

```
p = Path("data/file.txt")
```

A **Path object** represents a file or directory location.

11.3 Joining Paths

```
folder = Path("data")
file = folder / "raw" / "file.nc"
```

The `/` operator is used to join paths in a clean and readable way.

11.4 Checking Files and Directories

```
file.exists()
file.is_file()
file.is_dir()
```

These methods allow checking whether a path exists and what type it is.

11.5 Creating Directories

```
Path("data/processed").mkdir(parents=True, exist_ok=True)
```

`parents=True` creates all missing parent folders. `exist_ok=True` avoids errors if the folder already exists.

11.6 Listing Files with `glob`

```
folder = Path("data/raw")
files = folder.glob("*.nc")
```

`glob()` searches for files that match a pattern. For example, `*.nc` means all NetCDF files.

```
for f in files:
    print(f.name)
```

Each element returned by `glob()` is a Path object.

Chapter 12

Xarray

Xarray is a Python library for working with **multi-dimensional labeled data**. It is especially useful for scientific data such as weather, climate, and satellite data.

12.1 Importing Xarray

```
import xarray as xr
```

The alias `xr` is commonly used for xarray.

12.2 Opening Datasets

```
ds = xr.open_dataset("file.nc")
```

This loads a dataset from a NetCDF or GRIB file into an xarray object.

12.3 Dataset Structure

An xarray Dataset contains:

- **Dimensions:** axes like latitude, longitude, time
- **Coordinates:** labeled values for dimensions
- **Data variables:** actual data (e.g., u10, v10)

```
print(ds)
```

12.4 Accessing Variables

```
ds["u10"]
ds["v10"]
```

Variables are accessed like dictionary keys.

12.5 Converting to NumPy Arrays

```
u10 = ds["u10"].values
```

`.values` extracts the raw NumPy array from the xarray object.

12.6 Selecting Data with `sel`

```
subset = ds.sel(
    longitude=slice(2, 7),
    latitude=slice(52, 49)
)
```

`sel()` selects data using coordinate values (labels), not indices.

12.7 Index-Based Selection with `isel`

```
subset = ds.isel(latitude=0)
```

`isel()` selects data using integer indices.

12.8 Saving Datasets

```
ds.to_netcdf("output.nc")
```

This saves the dataset to a NetCDF file.

12.9 Key Idea

Xarray combines the power of NumPy arrays with labeled dimensions, making it ideal for structured scientific data.

Chapter 13

System and Communication Libraries

These libraries allow Python to interact with the operating system, networks, hardware devices, and external services. They are commonly used in backend and infrastructure software.

13.1 socket

The `socket` module enables low-level network communication.

```
import socket

s = socket.socket()
s.connect(("localhost", 8080))
s.send(b"Hello")
s.close()
```

13.2 pyserial

`pyserial` is used for serial communication with hardware devices.

```
import serial

ser = serial.Serial("/dev/ttyUSB0", baudrate=9600)
data = ser.readline()
ser.close()
```

13.3 InfluxDB

InfluxDB is a time-series database used for monitoring and metrics. Python clients allow writing and querying time-based data.

```
from influxdb_client import InfluxDBClient

client = InfluxDBClient(url="http://localhost:8086", token="TOKEN")
```

13.4 Paramiko

`paramiko` is used for SSH connections and remote command execution.

```
import paramiko

ssh = paramiko.SSHClient()
ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())
ssh.connect("server.com", username="user")
stdin, stdout, stderr = ssh.exec_command("ls")
ssh.close()
```

13.5 subprocess

The `subprocess` module allows running external system commands. It replaces older modules like `os.system`.

```
import subprocess

result = subprocess.run(
    ["ls", "-l"],
    capture_output=True,
    text=True
)

result.stdout
```